

CSE 144 Final Project Report

Transfer Learning Challenge

Adam Margulies

Spring 2026

1 Introduction

The goal of this project was to use a transfer learning approach and train a classifier on a 100-class image dataset with only a small number of labeled training examples per class. Transfer learning was appropriate for this challenge because a pretrained model can use visual features learned from a much larger dataset instead of learning all of the features from scratch. My final approach used a pretrained ViT-B/16 model, where I trained the classifier head first, and then fine-tuned the last two transformer blocks and used test-time augmentation for inference. With this the final model achieved a validation accuracy of 0.687 and a Kaggle public score of 0.66363.

2 Dataset

The dataset has 100 image classes, and about 10 training images per class with an unlabeled test set used for Kaggle submission. The data itself is organized into folders from 0 through 99, which I had to force label mapping so that folder 0 maps to label 0, folder 1 maps to label 1, and so on. For preprocessing all of the images were resized to 224 x 224 and normalized with ImageNet statistics. Training images also used random resizing crops, horizontal flips, rotations, and color jitter for augmentation.

3 Implementation

3.1 Model

The model used was a pretrained TorchVision ViT-B/16 model because it is strong for image classification and performed best in my experiments. The only architecture change in this final model was modifying the original ImageNet classifier head to a linear layer for 100 output classes. My fine-tuning strategy involved first training only this new classifier with the ViT backbone frozen, and then afterwards unfreezing the last two transformer encoder blocks and training some more with a smaller learning rate.

3.2 Training

The loss function I used was cross-entropy loss with label smoothing. I used the AdamW optimizer, and I used a ReduceLROnPlateau scheduler. The main hyperparameters were batch size 16, head learning rate 1e-3, encoder learning rate 1e-5, weight decay 1e-4, 15 epochs for the

first training session, and 20 epochs for the second. The training was done in google collab with T4 GPU, with PyTorch and TorchVision.

4 Experiments

I initially began with a pretrained ResNet model as a baseline to make sure that the data loading, label mapping, validation split, and Kaggle submission format were working as they should. Once I got that to work, I tried a pretrained ViT-B/16 model, then tested EfficientNet-B0 to gauge if a smaller model would do a better job at generalizing with a small sample size. However, the pretrained ViT did so much better that I went back to it to work on fine-tuning it. I also tested DenseNet, which appeared to overfit much less than the ViT model, but its validation accuracy did not reach the ViT model's validation accuracy. Based on these experiments I ended up choosing the fine-tuned ViT-B/16 as my final model, and continued to improve on it by implementing data augmentation, label smoothing, early stopping, and horizontal-flip test-time augmentation.

5 Results

The final fine-tuned ViT-B/16 model achieved a validation accuracy of 0.687 and a validation loss of 1.99. On Kaggle, using predictions generated from the best fine-tuned ViT checkpoint, my final public leaderboard score was 0.66363.

6 Discussion

The best-performing approach was the fine-tuned ViT-B/16 model because it combined strong pretrained visual features with limited fine-tuning for the small dataset. The main issue is overfitting, since each class had only a small number of training images. To try and combat this I used augmentation, label smoothing, early stopping, weight decay, and partial fine-tuning. The smaller models overfit less with DenseNet looking very good, but I could not get the validation accuracy up to as much as the ViT-B/16 model's validation accuracy. The biggest limitation to my approach is that the dataset was so small. Future improvements could focus on testing more augmentation approaches or softer fine tuning, or just choosing to use a smaller model.

7 Reproducibility

I used a fixed random seed of 42 for Python, NumPy, and PyTorch. I also set a deterministic CuDNN behaviour when CUDA was available. The code can be run by installing the required packages with `pip install -r requirements.txt`, and then running python [train.py](#) to train the model. To generate the Kaggle submission with the provided model weights, first place the checkpoint at `outputs/best_vit_ft.pth`, and then inference can be run with python [predict.py](#). This creates the final submission file at `outputs/submission_vit.csv`.

8 Team Contributions

I worked solo so I did all of the work.

9 References

1. CSE 144 Lecture Slides, Transfer Learning and Image Classification, Spring 2026.
2. TorchVision Documentation. “Models and Pre-trained Weights.” PyTorch/TorchVision.
3. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” ICLR, 2021.
4. Alexander Otchenashenko, “9 Setting Up a Pretrained Model with TorchVision”, Youtube, February 9, 2023.